

PocketSmart AI: Your Smart Budget & Recommendation Assistant

(PROJECT-DOCUMENTATION)

Project Description:

PocketSmart AI harnesses Generative AI to provide budget-aware planning and personalized recommendations for everyday needs. The platform gives users a structured way to **plan Home Interior, Party, and Jewelry requirements within a stated budget**, while reducing the effort of comparing many options manually. The application is implemented with **FastAPI and Jinja2** and provides **authentication, session-aware recommendation history, structured Pydantic validation, SQLite persistence, Gemini-powered recommendations, optional jewelry outfit-image analysis, and safe search/demo links for external platforms**. When Gemini is unavailable, the application uses a clearly labelled fallback so the student project remains runnable without pretending that external inventory is live.

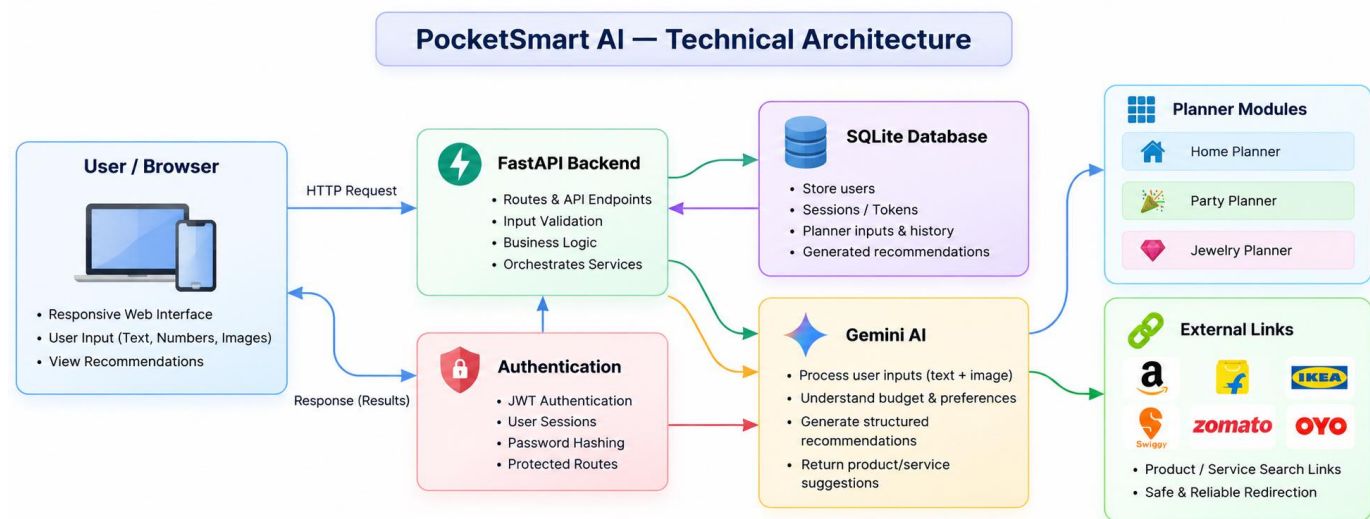
Scenarios:

Scenario 1: A user planning a home interior enters a **total budget** and **selects rooms, lighting, ceiling fans, furniture/table quantities and preferences**. PocketSmart AI produces an **allocation, recommended items, indicative/demo prices, remaining budget and relevant shopping/search links**.

Scenario 2: A user organizing a party **enters the total budget**, number of guests, event type, venue, catering, decoration and entertainment preferences. The system **divides the budget across practical categories** and returns a structured recommendation that respects the requested spending limit.

Scenario 3: A user planning jewelry selects an occasion and style, **describes the outfit**, and may upload a **JPG, PNG or WEBP** outfit image. The AI uses the **supplied context to suggest jewelry choices**, while the application keeps recommendations within the selected budget and stores the result in history.

Technical Architecture:



Description: PocketSmart AI uses a modular web architecture in which the responsive Jinja2 frontend communicates with a FastAPI backend. The backend validates inputs, manages authentication and sessions, persists users and recommendation history in SQLite, and calls Gemini through the current google-genai SDK when a valid API key is configured. Planner modules generate structured Home, Party and Jewelry recommendations, while external platforms are represented by safe search/demo links.

(Note: Because this project uses SQLite persistence, the database entities are part of the architecture: users, sessions and recommendations. The implementation does not require a separate database server for the student deployment.)

Pre-requisites:

- FastAPI / Uvicorn Knowledge: FastAPI application structure, routing and server execution
- Gemini API Familiarity: Google Gemini / google-genai SDK configuration
- HTML, CSS and JavaScript Skills: Jinja2 templates and responsive browser-side interactions
- Python Programming Proficiency: application modules, validation and exception handling
- SQLite / Data Persistence: basic relational tables and session/history storage
- Version Control with Git: repository management and safe secret handling
- Development Environment Setup: Python virtual environment and requirements installation

Project Workflow:

Milestone 1: Model Selection and Architecture

- Activity 1.1: Configure Gemini API credentials securely through .env and the application settings.
- Activity 1.2: Select a Gemini model through GEMINI_MODEL so model choice can be changed without editing application logic.
- Activity 1.3: Define the architecture covering Jinja2 frontend, FastAPI backend, authentication, SQLite persistence and Gemini integration.
- Activity 1.4: Set up the Python virtual environment and install the project dependencies.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop Home Interior, Party and Jewelry planners with budget-aware structured outputs.
- Activity 2.2: Implement FastAPI routes, Pydantic validation, persistence and graceful Gemini fallback.

Milestone 3: Backend Development

- Activity 3.1: Build the main FastAPI application logic, authentication/session handling, recommendation generation and history retrieval.

Milestone 4: Frontend Development

- Activity 4.1: Design responsive blue/white Jinja2 pages for registration, login, dashboard and planner workflows.
- Activity 4.2: Add browser-side interactions for form submission, result presentation, uploads and history navigation.

Milestone 5: Deployment and Verification

- Activity 5.1: Prepare the application for local deployment and execute automated tests.
- Activity 5.2: Deploy the working application to Vercel and verify the public routes.

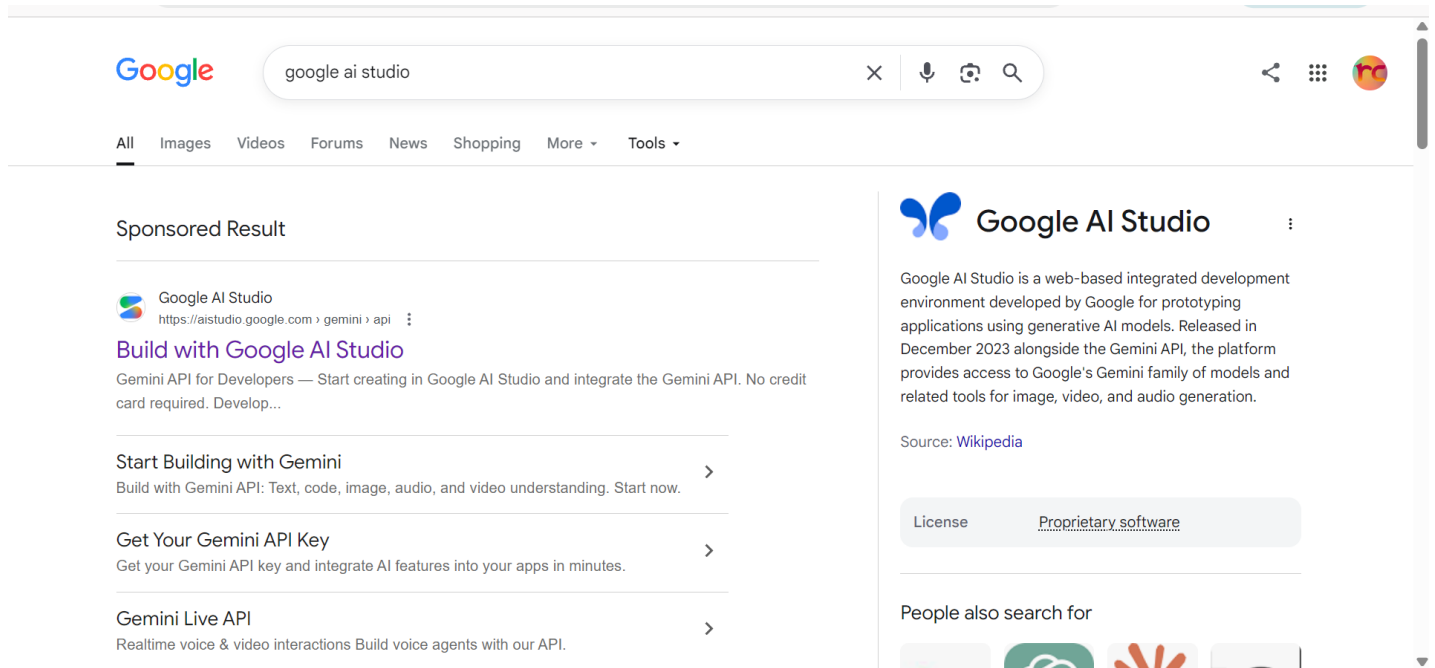
Milestone 1: Model Selection and Architecture

In this milestone, the project focuses on selecting and configuring the generative AI component used for budget-aware recommendations. The architecture is designed so that Gemini configuration remains externalized, while the FastAPI application controls validation, structured prompting, persistence and safe presentation of results.

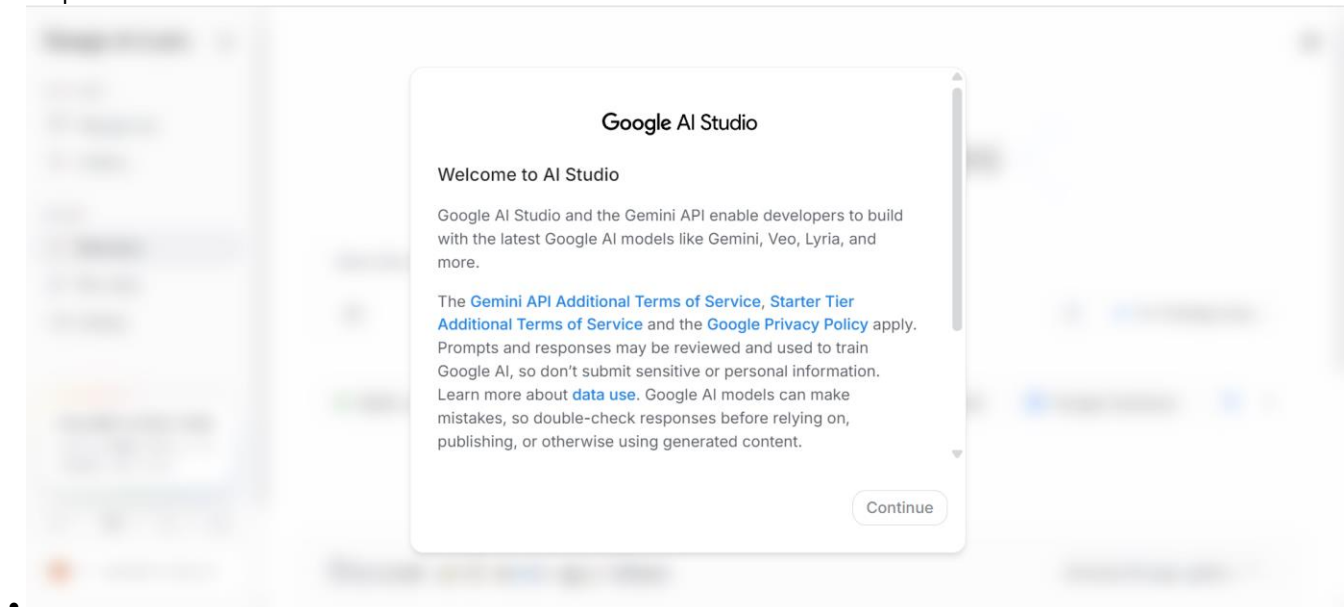
Activity 1.1: Configure the Gemini API Key

Before the application can communicate with the Gemini service, a valid API key must be available. The project reads the credential from the environment rather than hard-coding it into Python source files.

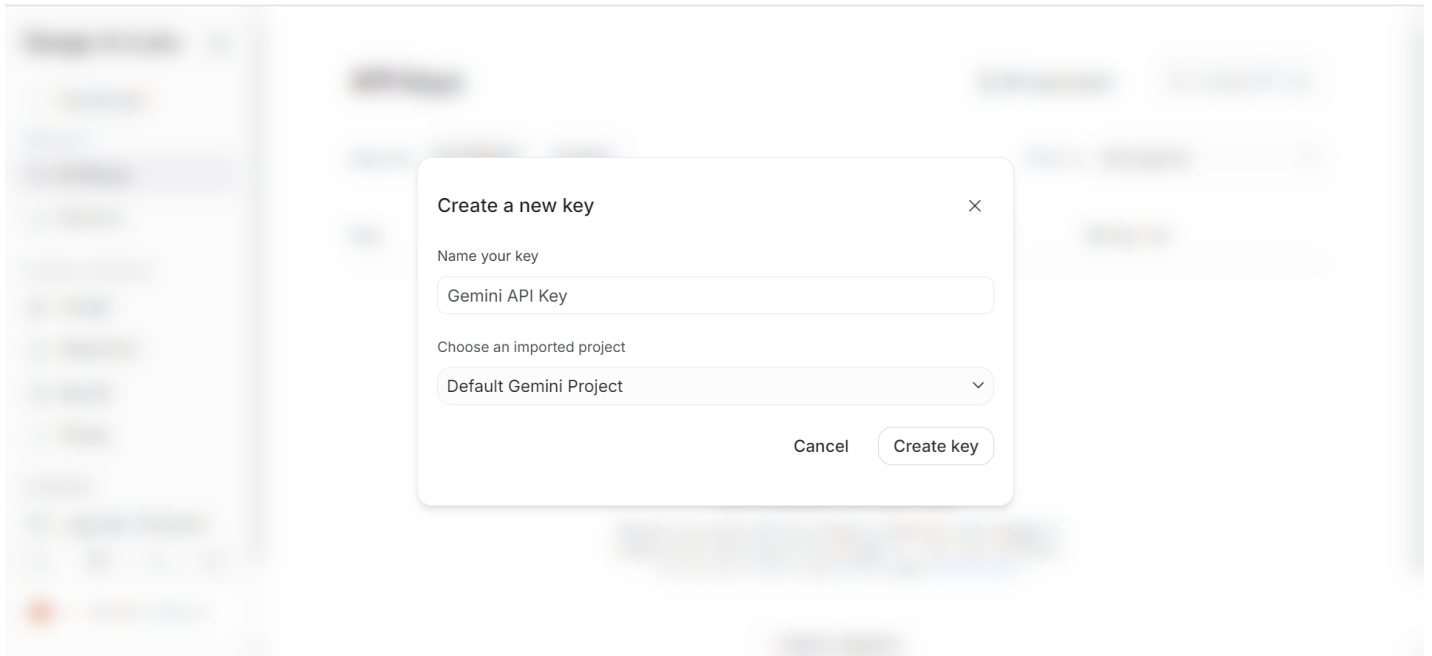
- Step 1 — Obtain a Gemini API key using Google's current Gemini / AI Studio developer tooling.



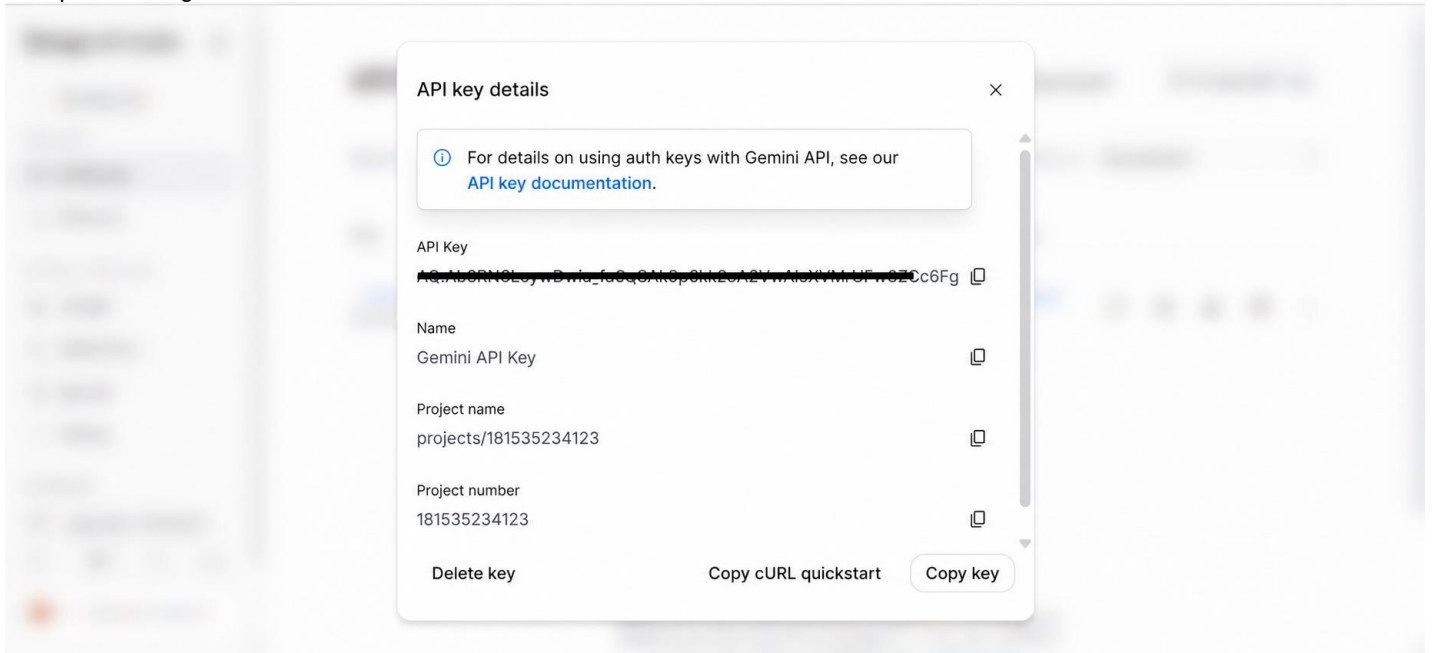
- Step 1.1: Click with Get Started and continue



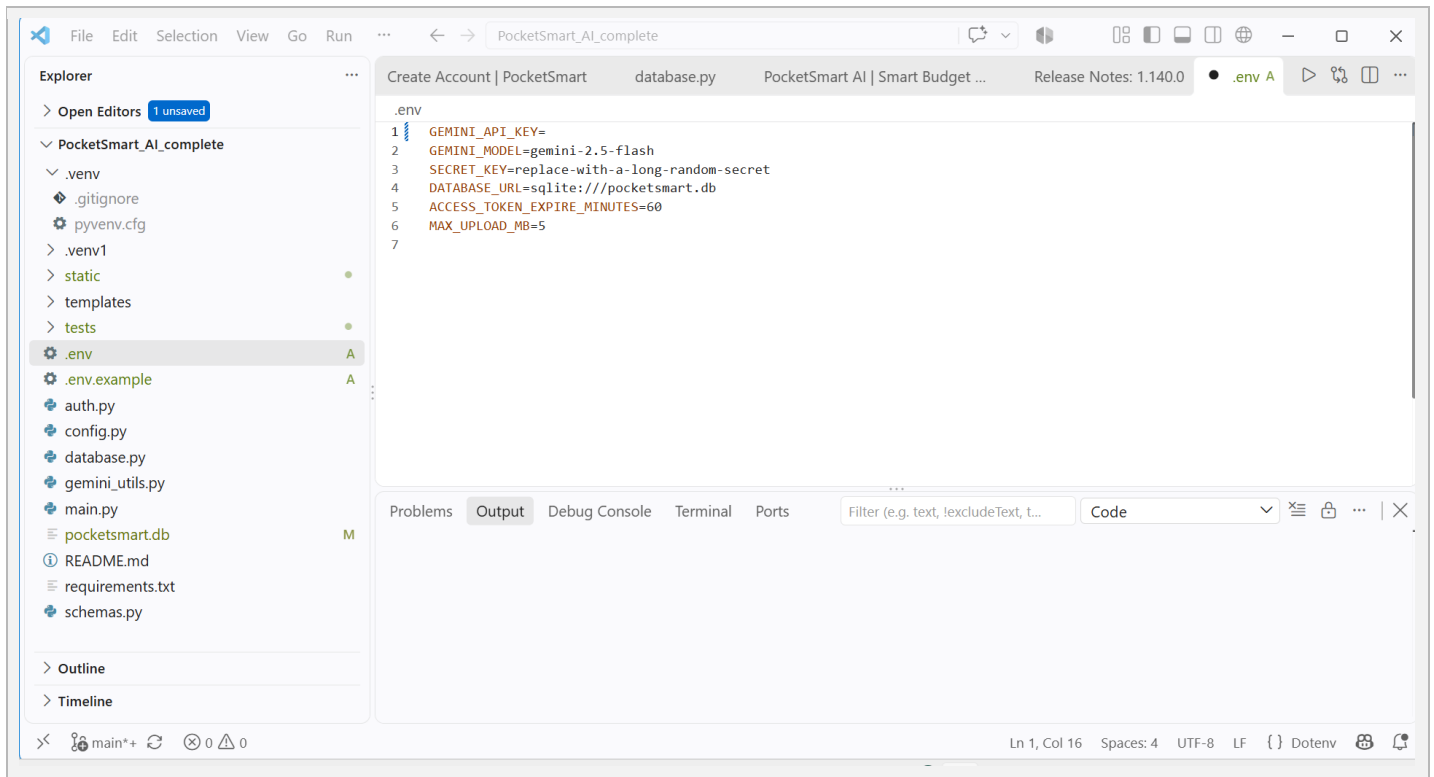
- Step 1.2: Navigate to Get API key and click Create API Key



- Step 1.3: It will generate an API KEY



- Step 2 — Create or open the .env file in the PocketSmart project root.
- Step 3 — Add GEMINI_API_KEY with the generated key.
- Step 4 — Set GEMINI_MODEL to the model you intend to use, for example gemini-2.5-flash when supported by the configured account.



- Step 5 — Set a long random SECRET_KEY for session/JWT-related application security.
- Important — Never Commit Your API Key: keep .env outside Git tracking and use .env.example for the safe configuration template.

Activity 1.2: Environment Configuration and Secret Handling

Step 1 — Add the application settings: the project keeps runtime configuration in environment variables so the same source code can be used locally and in deployment.

```
.env
1 GEMINI_API_KEY=gemini-api-key
2 GEMINI_MODEL=gemini-2.5-flash
3 SECRET_KEY=replace-with-a-long-random-secret
4 DATABASE_URL=sqlite:///pocketsmart.db
5 ACCESS_TOKEN_EXPIRE_MINUTES=60
6 MAX_UPLOAD_MB=5
7
```

Step 2— Verify configuration: the FastAPI application loads the settings at startup and the Gemini helper reads the configured model and key. If the key is missing or the service is unavailable, the application switches to its clearly labelled demo fallback.

```
# Configuration concept
settings = get_settings()
model_name = settings.GEMINI_MODEL
api_key = settings.GEMINI_API_KEY
# Gemini is used only when a valid key is available.
```

Step 3 — Protect the repository: .gitignore should exclude .env, the SQLite database used during local development, virtual-environment files, caches and uploaded/generated runtime artifacts where appropriate.

```
1  .venv/
2  __pycache__/
3  .pytest_cache/
4  .env
5  *.pyc
6  pocketSMART.db
7  static/uploads/*
8  !static/uploads/.gitkeep
9  .env.example
10 .venv1/
```

+

Step 4 — Do not place credentials in templates, JavaScript files, README examples or screenshots.

Activity 1.3: Research and Select the Appropriate Generative AI Model

- **Understand the Project Requirements:** recommendations must be structured, budget-aware and suitable for Home Interior, Party and Jewelry use cases.
- **Review Gemini Model Documentation:** examine supported models, context limits, latency, structured-output capabilities and account availability.
- **Evaluate Model Behaviour:** verify recommendation relevance, response consistency, budget compliance and practical latency.
- **Select Through Configuration:** use GEMINI_MODEL so the deployed application is not locked to one hard-coded model name.

Activity 1.4: Define the Architecture of the Application

- **Draft an Architectural Diagram:** represent the browser/Jinja2 layer, FastAPI routes, authentication, SQLite storage, Gemini integration and planner modules
- **Detail Frontend Functionality:** users register/login, open a planner, enter budget and preferences, optionally upload an outfit image, then review structured results.
- **Outline Backend Responsibilities:** validate inputs with Pydantic schemas, enforce positive budgets/quantities/prices, construct the AI request, normalize generated totals and persist the recommendation.
- **Describe AI Integration Points:** the Gemini helper receives structured planner context and returns a recommendation that is normalized before presentation.

Activity 1.5: Set Up the Development Environment

- Install Python: use a supported Python 3.x installation with pip.
- Create a Virtual Environment: isolate PocketSmart dependencies from the system Python installation.

```
C:\Projects>python -m venv .venv
C:\Projects>.venv\Scripts\activate
(.venv) C:\Projects>python -m pip install --upgrade pip
(.venv) C:\Projects>python -m pip install -r requirements.txt
```

If the python command is not recognized, use the full path to the installed Python executable.

```
C:\Program Files\Python313\python.exe -m venv .venv
```

Set Up Application Structure: create the main Python modules, Jinja2 templates, static assets, upload folder and automated tests described in the project structure.

```
PocketSmart/
├── main.py
├── gemini_utils.py
├── database.py
├── schemas.py
├── auth.py
├── config.py
├── requirements.txt
├── .env.example
├── .gitignore
├── README.md
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   ├── dashboard.html
│   ├── home_planner.html
│   ├── party_planner.html
│   ├── jewelry_planner.html
│   ├── history.html
│   ├── recommendation_details.html
│   └── recommendation_result.html
├── static/
│   ├── css/styles.css
│   ├── js/app.js
│   └── uploads/
└── tests/
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Budget and Recommendation Features

- **Home Interior Planner:** accepts rooms, lighting, ceiling fans, furniture/table quantities and preferences and returns a budget-aware item allocation.
- **Party Planner:** accepts budget, guests, event type, venue, catering, decoration and entertainment details and produces category-level planning.
- **Jewelry Planner:** accepts occasion, style, outfit description and optional JPG/PNG/WEBP image context to generate matching recommendations.
- **Recommendation History:** stores generated results so authenticated users can revisit earlier recommendations and open details.

Activity 2.2: Implement the FastAPI Backend

- **Define Routes:** create GET routes for pages and POST routes for login, registration, logout and each planner generation endpoint.
- **Process User Input:** receive form data and uploaded images, validate them with structured schemas and reject invalid budgets, quantities or unsupported uploads.
- **Integrate AI Calls:** send planner context to Gemini when configured and use the project fallback when the AI service is unavailable.
- **Persist Results:** save authenticated-user sessions and recommendation history in SQLite.

```
POST /generate-home
POST /generate-party
POST /generate-jewelry
GET /history
GET /recommendations-details/{recommendation_id}
```

Milestone 3: Backend Development

Milestone 3 focuses on creating and refining the FastAPI application logic, which serves as the backbone of PocketSmart AI. The backend connects the user-facing planners to authentication, validation, Gemini generation, SQLite persistence and structured result pages.

Activity 3.1: Writing the Main Application Logic

Define the Core Routes in main.py:

- **/** — **Home page** route that renders the landing page.
- **/dashboard** — Protected dashboard for authenticated users.
- **/home-planner**, **/party-planner**, **/jewelry-planner** — Planner pages.
- **/login** and **/register** — Authentication pages.
- **/generate-home**, **/generate-party**, **/generate-jewelry** — Recommendation generation endpoints.
- **/history** and **/recommendations-details/{recommendation_id}** — Stored recommendation access.

Representative FastAPI structure:

```
105 def logout(request: Request):
106     try: revoke_session(decode_token(token)["jti"])
107     except Exception: pass
108     r=JSONResponse({"message":"Logged out."}); clear_auth_cookie(r); return r
109
110 @app.post("/token")
111 def token_alias(payload: LoginInput): return login(payload)
112
113 @app.get("/dashboard", response_class=HTMLResponse)
114 def dashboard(request: Request, user=Depends(get_current_user)):
115     rows=get_recommendations(user["id"],6)
116     return render(request,"dashboard.html",user=user,recent=rows)
117
118 @app.get("/home-planner", response_class=HTMLResponse)
119 def home_planner(request: Request, user=Depends(get_current_user)): return render(request,"home_planner.html",user=user)
120 @app.get("/party-planner", response_class=HTMLResponse)
121 def party_planner(request: Request, user=Depends(get_current_user)): return render(request,"party_planner.html",user=user)
122 @app.get("/jewelry-planner", response_class=HTMLResponse)
123 def jewelry_planner(request: Request, user=Depends(get_current_user)): return render(request,"jewelry_planner.html",user=user)
124
125 @app.post("/generate-home")
126 def generate_home_route(payload: HomeInput, user=Depends(get_current_user)):
127     result=generate_home(payload); rid=create_recommendation(user["id"],"home",json.dumps(payload.model_dump()))
128     return {"id":rid,"result":result.model_dump()}
129
130 @app.post("/generate-party")
131 def generate_party_route(payload: PartyInput, user=Depends(get_current_user)):
132     result=generate_party(payload); rid=create_recommendation(user["id"],"party",json.dumps(payload.model_dump()))
133     return {"id":rid,"result":result.model_dump()}
134
135 @app.post("/generate-jewelry")
136 async def generate_jewelry_route(budget: float=Form(...), occasion: str=Form(...), style_preference: str=Form(...),
137     if budget<=0: raise HTTPException(422,"Budget must be greater than zero.")
138     image_bytes=None; mime=None; saved_name=None
139     if image and image.filename:
140         allowed=("image/jpeg","image/png","image/webp")
141         if image.content_type not in allowed: raise HTTPException(400,"Upload a JPG, PNG, or WEBP image.")
142         data=await image.read()
143         if len(data)>settings.max_upload_mb*1024*1024: raise HTTPException(400,f"Image must be smaller than {settings.max_upload_mb} MB")
```

Milestone 3: Backend Development — Validation and Route Handling

Set Up Route Handlers for Each Planner

For each planner endpoint, the application reads structured input, validates it, builds the AI request, handles service failures, normalizes generated results and stores the final recommendation for the current session/user.

```
# Conceptual flow used by the planner endpoints
validate_request()
check_authenticated_session()
if gemini_available:
    result = generate_with_gemini(context)
else:
    result = demo_fallback(context)
result = enforce_budget(result, requested_budget)
save_recommendation(user_id, result)
return render_result(result)
```

Input Validation

- Budget values must be positive.
- Quantities must be valid positive values where applicable.
- Generated item prices are normalized so displayed totals do not exceed the requested budget.
- Jewelry image uploads are limited by configured size and supported image types.
- Authentication-protected pages are inaccessible without a valid session.

Session and History

Authenticated users can open the History page, select a saved recommendation, and view its details. Logging out invalidates the authenticated session path and protected pages redirect or reject access until login occurs.

Define Prompt Engineering Helpers

PocketSmart AI uses structured planner context rather than a single free-form text prompt. The prompt contains the budget, domain, user preferences and expected response structure so the generated recommendation can be validated and displayed consistently.

Planner Context

```
planner_context = {
    "domain": "home | party | jewelry",
    "budget": budget,
    "preferences": preferences,
    "constraints": constraints,
    "image_context": optional_image_information
}
```

Structured Recommendation Requirements

- Return practical categories/items relevant to the selected planner.
- Include indicative/demo pricing when real-time inventory is unavailable.
- Keep the recommendation within the user's requested budget after backend normalization.
- Provide concise reasons or notes that help the user understand each recommendation.
- Return data in a predictable structure that can be validated before rendering.

Model Configuration

```
GEMINI_MODEL = settings.GEMINI_MODEL
# Change the model through .env rather than editing application code.
```

The project documentation intentionally avoids claiming that Amazon, Flipkart, IKEA, Swiggy, Zomato or OYO are live inventory integrations. Their links are safe search/demo links only.

Integrate the Gemini AI Responses in Each Function

The Gemini helper is responsible for communicating with the configured model and returning structured recommendation content. The backend treats the AI response as untrusted input: it validates the structure and applies budget normalization before displaying the result.

```
# Simplified integration flow
response = gemini_client.generate_content(prompt)
result = parse_structured_response(response)
validated = RecommendationSchema.model_validate(result)
validated = enforce_budget(validated, requested_budget)
return validated
```

Graceful Fallback

When GEMINI_API_KEY is empty, invalid, rate-limited or temporarily unavailable, the application uses clearly labelled demo fallback recommendations. This allows the application to remain testable without representing the fallback as live market data.

Validation Before Frontend Rendering

- Confirm required recommendation fields exist.
- Validate the result with Pydantic models.
- Normalize totals against the requested budget.
- Attach safe search links where applicable.
- Persist the final recommendation and render the result page.

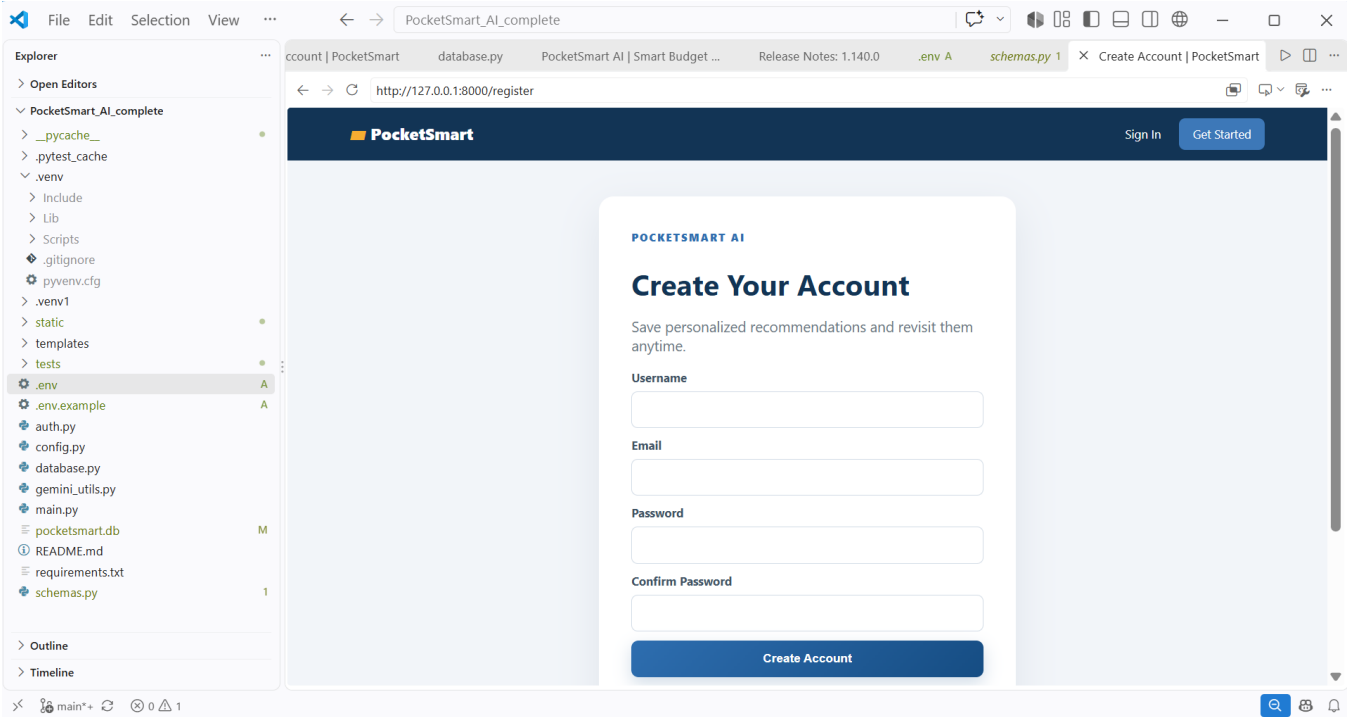
```
GET /startup
GET /session-info
GET /session-data
GET /history-data
```

Milestone 4: Frontend Development

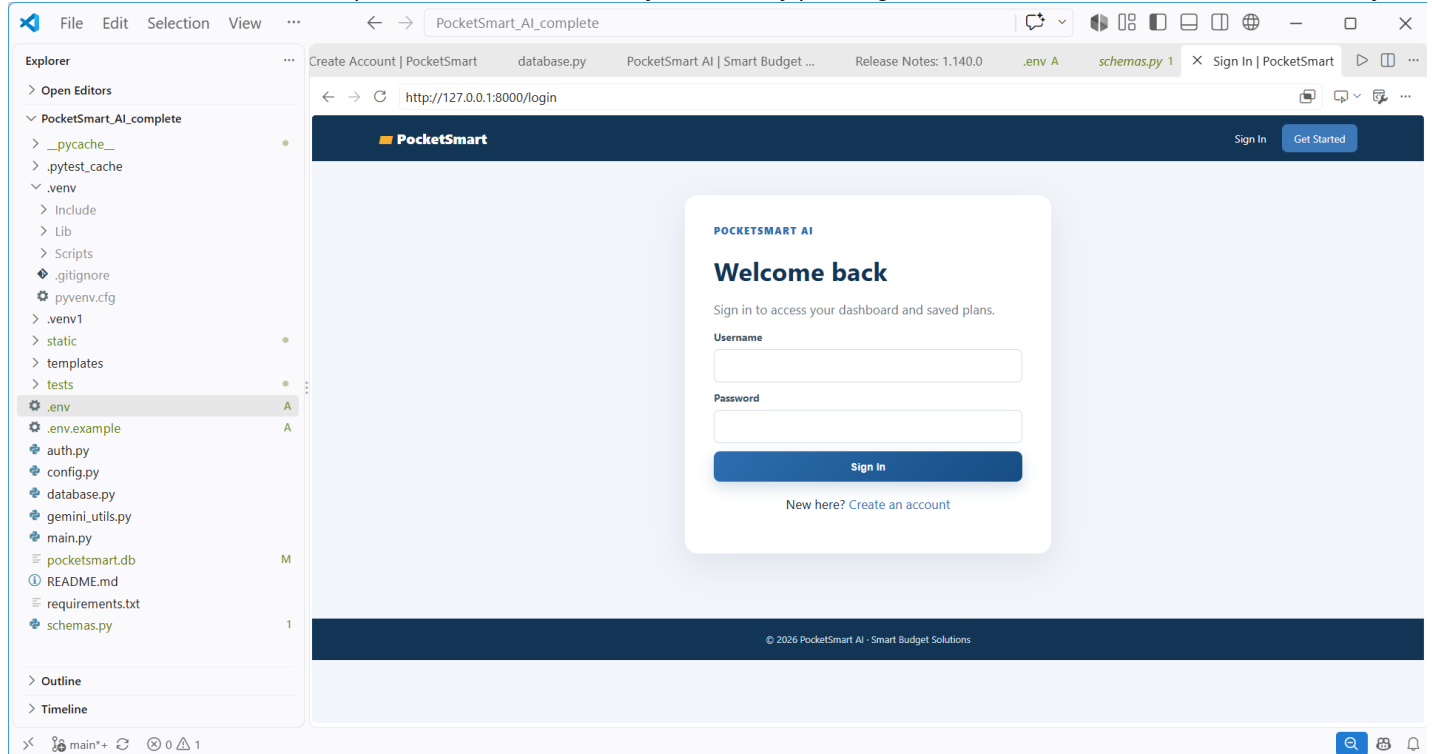
Milestone 4 focuses on developing a responsive and visually consistent blue/white interface for PocketSmart AI. Jinja2 templates provide the page structure while CSS and JavaScript handle responsive presentation and interactive browser behaviour.

Activity 4.1: Designing and Developing the User Interface

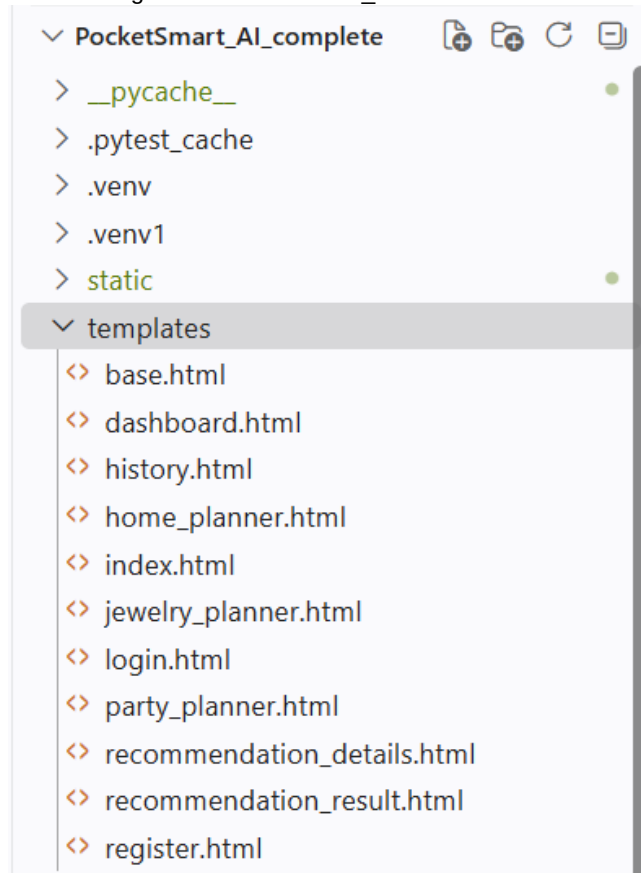
- Base Template: templates/base.html provides the common page structure, navigation and shared styling.
- Authentication Pages: login.html and register.html provide clear entry points for user accounts.



- **Dashboard:** dashboard.html exposes the main Home, Party and Jewelry planning workflows and recent recommendation history.



- **Planner Templates:** home_planner.html, party_planner.html and jewelry_planner.html collect domain-specific budget and preference information.
- **Result Pages:** recommendation_result.html and recommendation_details.html present structured AI output, totals and links.



Responsive Layout Using CSS

- Use the static/css/styles.css stylesheet for the blue/white visual system and responsive layouts.
- Use cards, form sections and clear action buttons to keep budget inputs and recommendations easy to scan.
- Use media queries so planner forms and recommendation cards remain usable on smaller screens.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

The browser-side application logic in static/js/app.js supports interactive form behaviour, result handling and small usability improvements without moving API secrets into the client.

Handle Form Submission

```
async function submitPlanner(formData, endpoint) {
  const response = await fetch(endpoint, {
    method: "POST",
    body: formData
  });
  if (!response.ok) throw new Error("Request failed");
  return response;
}
```

Render Results Dynamically

- Display budget allocation and recommendation categories in structured cards/tables.
- Show remaining budget after normalized item totals.
- Present safe external search links as navigation actions rather than claiming live stock.
- For Jewelry Planner, preserve the uploaded-image workflow while keeping file validation on the server.

Restore UI State After Generation

Loading indicators and button state are restored after successful or failed requests so the user can submit another planning request without refreshing the page.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying PocketSmart AI locally for verification and then publishing the application through the configured Vercel deployment. The same environment-variable approach is used in both environments so secrets are not embedded in source code.

Activity 5.1: Preparing the Application for Local Deployment

- Open the PocketSmart folder in VS Code and open a new terminal.
- Create and activate a virtual environment.

```
C:\Projects\PocketSmart>python -m venv .env
C:\Projects\PocketSmart>.env\Scripts\activate
(.env) C:\Projects\PocketSmart>python -m pip install -r requirements.txt
```

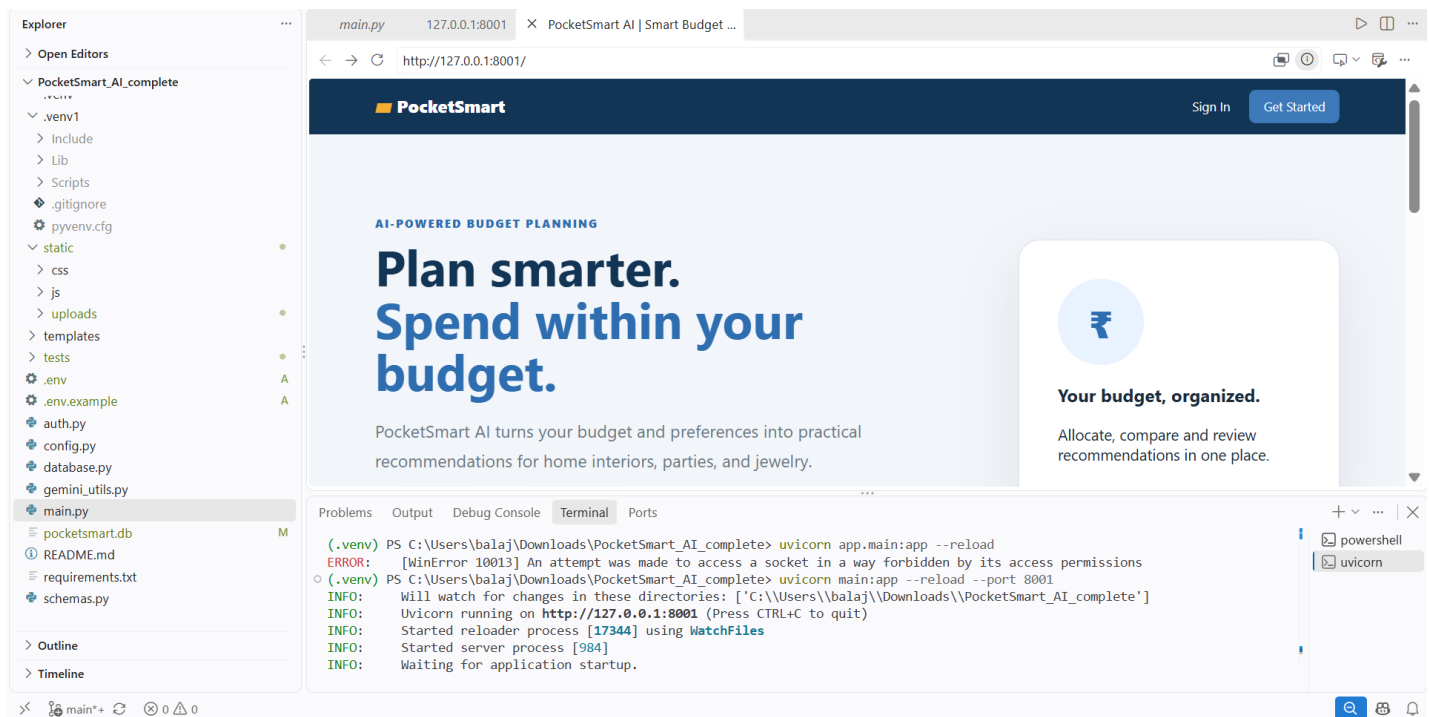
Configure Environment Variables

```
copy .env.example .env
# Then edit .env and set GEMINI_API_KEY and other runtime values.
```

Start the FastAPI Development Server

```
(.env) C:\Projects\PocketSmart>python -m uvicorn main:app --host 127.0.0.1 --port 8000
```

Once running, open <http://127.0.0.1:8000> in a browser.



Activity 5.2: Local Testing and Verification

Automated Test Verification

```
(.venv) C:\Projects\PocketSmart>python -m pytest -q
```

The automated test suite verifies the application's core behaviours and should be run before deployment.

```
(.venv) PS C:\Users\balaj\Downloads\PocketSmart_AI_complete> python -m pytest -q
C:\Users\balaj\Downloads\PocketSmart_AI_complete\.venv\Lib\site-packages\jwt\api_jwt.py:370: InsecureKeyLengthWarning: The HMAC key is 15 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
  decoded = self.decode_complete(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
7 passed, 15 warnings in 22.62s
(.venv) PS C:\Users\balaj\Downloads\PocketSmart_AI_complete>
```

Syntax / Import Smoke Check

```
(.venv) C:\Projects\PocketSmart>python -m compileall .
```

Manual Test Procedure

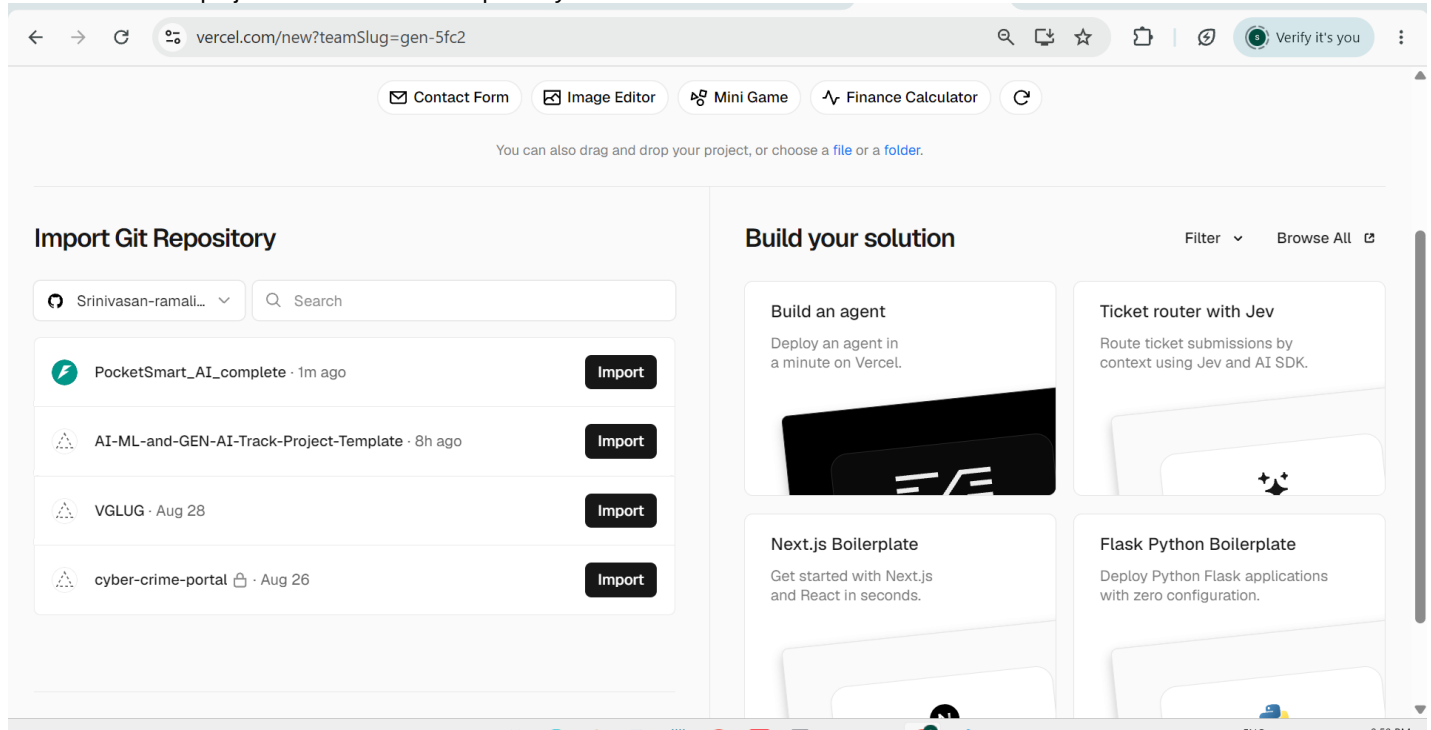
1. Open /register and create an account.
2. Confirm the dashboard opens after registration.
3. Open Home Budget Planner and submit a budget; inspect allocation, items, prices, remaining budget and links.
4. Open Party Budget Planner and submit a guest/event scenario.
5. Open Jewelry Budget Planner and submit an occasion/style; optionally upload a JPG/PNG/WEBP image.
6. Open History and select a recommendation to view its details.
7. Log out and verify protected pages redirect or reject access until login occurs.

Activity 5.3: Public Deployment via Vercel

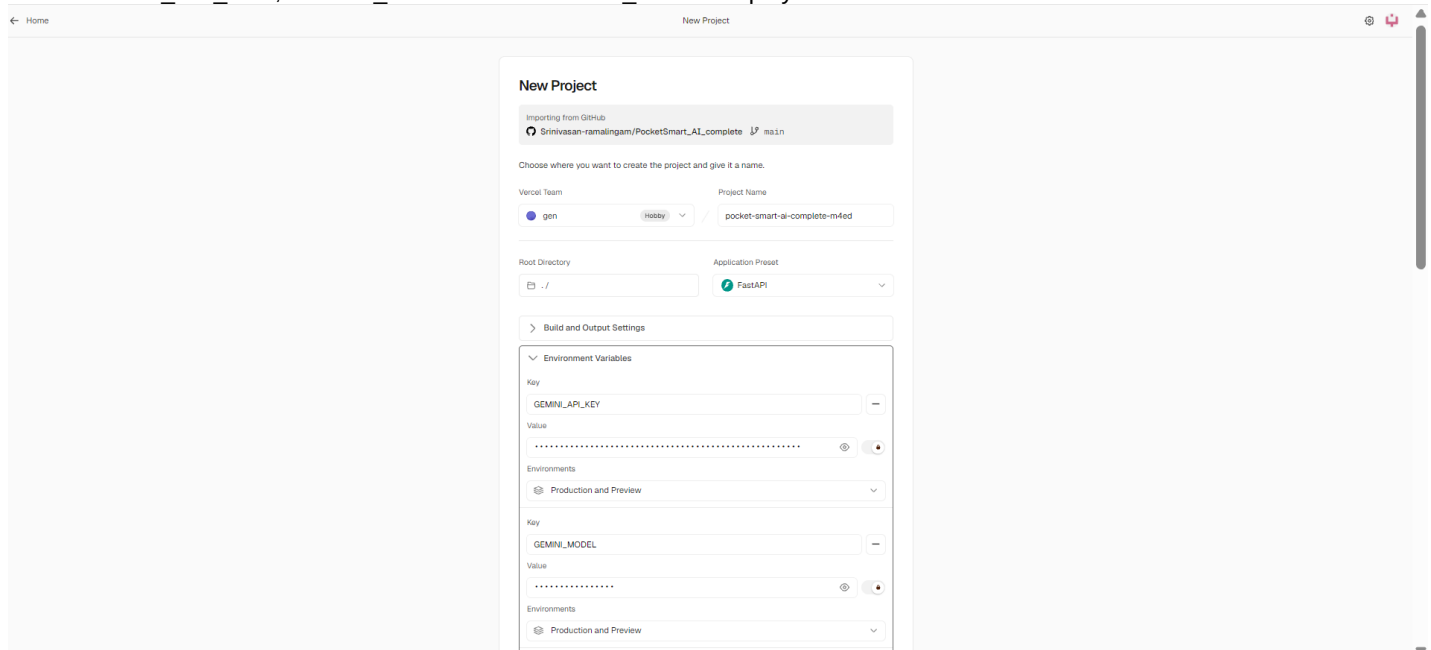
The current project deployment is hosted through Vercel. The repository contains the FastAPI application and its deployment configuration. Environment variables must be configured in the deployment platform rather than committed to Git.

Deployment Preparation

- Push the tested project source to the Git repository.



- Connect the repository to Vercel and configure the project as required by the deployment setup.
- Add GEMINI_API_KEY, GEMINI_MODEL and SECRET_KEY as deployment environment variables.



- Deploy and open the generated public URL.

The screenshot shows the Vercel deployment page for a project named 'pocket-smart-ai-complete'. The left sidebar contains navigation links: Overview, Deployments (selected), Logs, Analytics, Speed Insights, Observability, Firewall, CDN, Environment Variables, Domains, Connect, Integrations, Storage, Flags, Agent, and a user profile for 'srinivasanramaling...'. The main content area displays 'Deployment Details' for a deployment created 2 hours ago by 'Srinivasan-rama...'. The status is 'Ready Stale'. The duration is '19s 2h ago'. The environment is 'Production'. The domains listed are 'pocket-smart-ai-complete-git-main-gen-5fc2.vercel.app' and 'pocket-smart-ai-complete-1kaqvapo-gen-5fc2.vercel.app'. The source is 'main' with a commit hash 'd507dac'. Below the deployment details, there are sections for 'Deployment Settings' (with 2 recommendations), 'Build Logs', 'Deployment Summary', 'Deployment Checks', and 'Assigning Custom Domains'. A 'Visit' button is visible in the top right of the deployment details section.

- Verify the home page, authentication flow, planner routes and result pages.

The screenshot shows the home page of the 'PocketSmart' application. The header is dark blue with the 'PocketSmart' logo on the left and 'Sign In' and 'Get Started' buttons on the right. The main content area has a light blue background. On the left, there's a section titled 'AI-POWERED BUDGET PLANNING' with the headline 'Plan smarter. Spend within your budget.' and a sub-headline 'PocketSmart AI turns your budget and preferences into practical recommendations for home interiors, parties, and jewelry.' Below this are 'Get Started' and 'Learn / Sign In' buttons. On the right, there's a white card with a blue circle containing a rupee symbol (₹) and the text 'Your budget, organized.' followed by 'Allocate, compare and review recommendations in one place.'

Current Public Deployment

<https://pocket-smart-ai-complete-6l3n-7cl40maem-gen-5fc2.vercel.app/>

Repository

https://github.com/Srinivasan-ramalingam/PocketSmart_AI_complete

The public deployment should be treated as a student/demo deployment. Gemini availability depends on the configured API key, selected model and account limits.

Deployment Verification and Important Notes

Public Demo

Share the deployed URL with evaluators so they can open the application directly in a browser without installing the project locally.

Public URL:
https://pocket-smart-ai-complete-6l3n-7cl40maem-gen-5fc2.vercel.app

Project Demonstration

https://www.loom.com/share/b9ce320e4df046db8dc89ec8aecd4999

Important Notes:

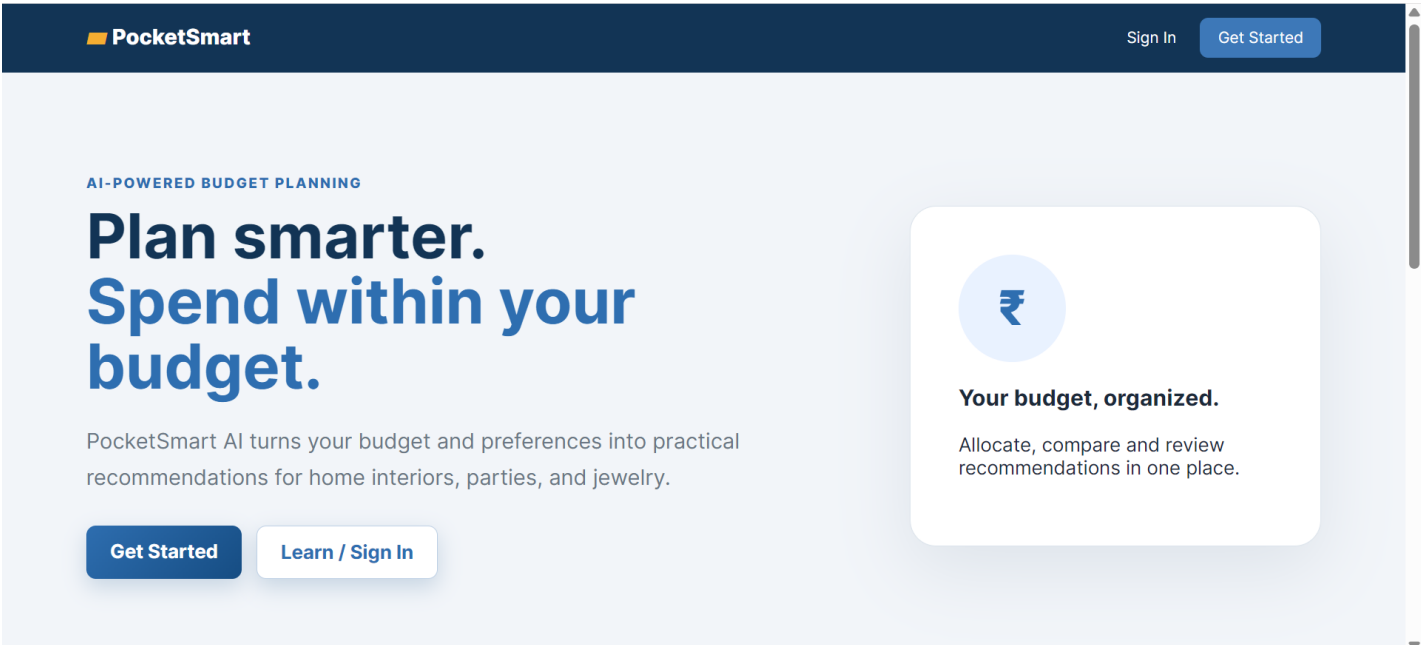
- The public URL is the current hosted deployment and may change if deployment configuration is changed.
- Gemini-powered responses require a valid deployment environment variable and an available model/account.
- Search links to Amazon, Flipkart, IKEA, Swiggy, Zomato and OYO are not live inventory APIs or scraping integrations.
- Demo prices are illustrative and should not be interpreted as current market quotations.
- SQLite is suitable for the student/demo application; production multi-user hosting would normally use a managed database.

Backend Routes

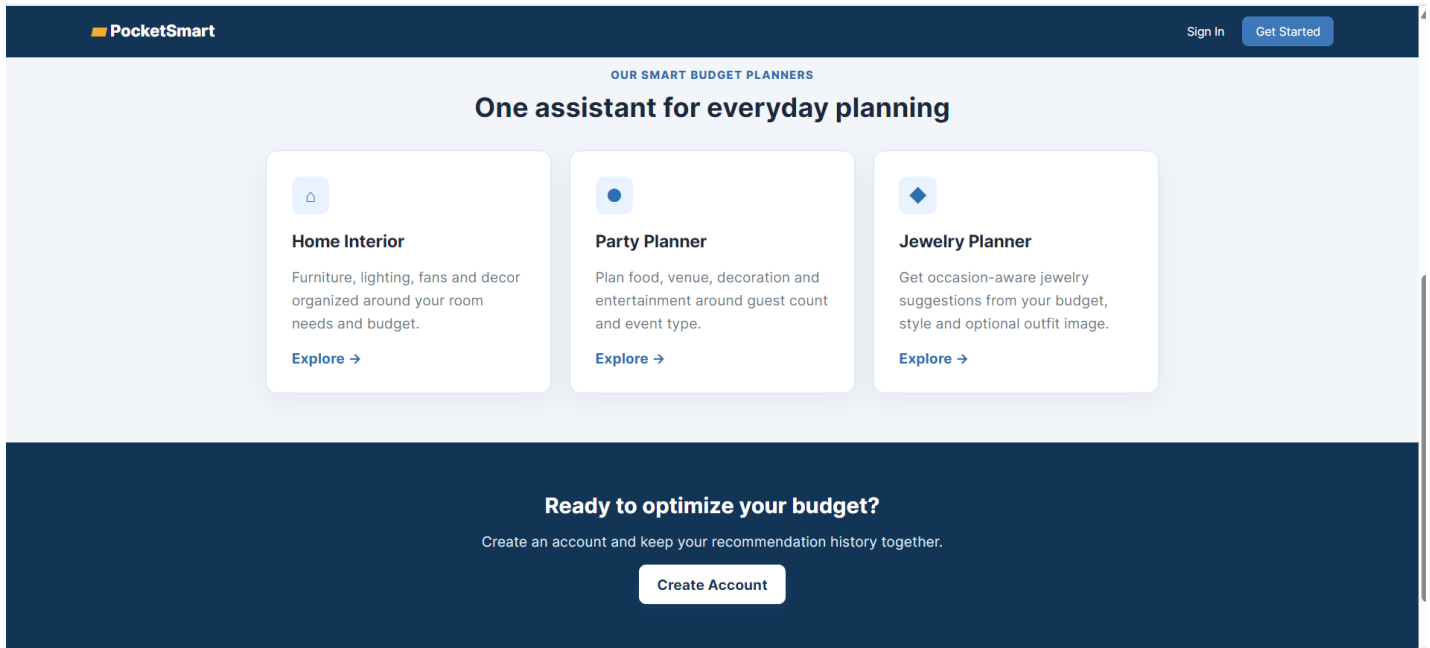
Method	Route	Purpose
GET	/	Home page
GET	/dashboard	Protected dashboard
GET	/home-planner	Home planner
GET	/party-planner	Party planner
GET	/jewelry-planner	Jewelry planner
POST	/login, /register, /logout	Authentication
POST	/generate-home	Home recommendation
POST	/generate-party	Party recommendation
POST	/generate-jewelry	Jewelry recommendation
GET	/history	Recommendation history
GET	/recommendations-details/{recommendation_id}	Recommendation details
GET	/startup	Startup health check

Exploring the Website’s Web Pages:

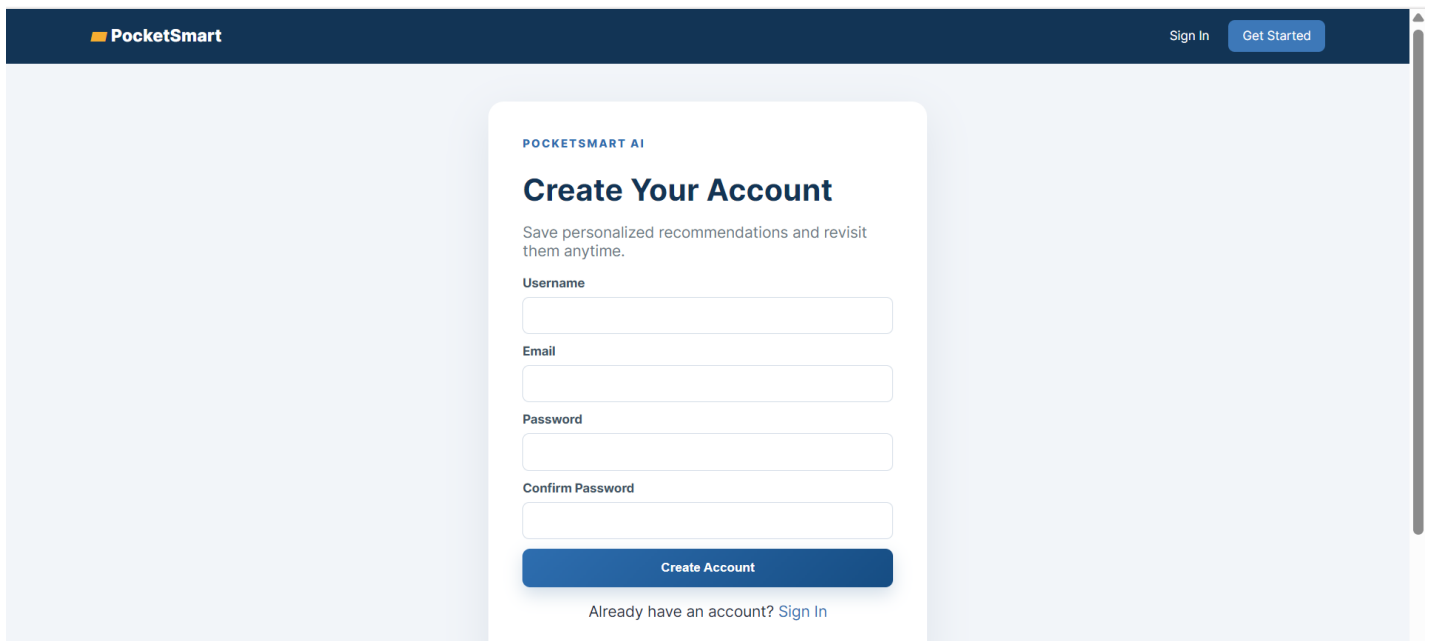
Home / Landing Page



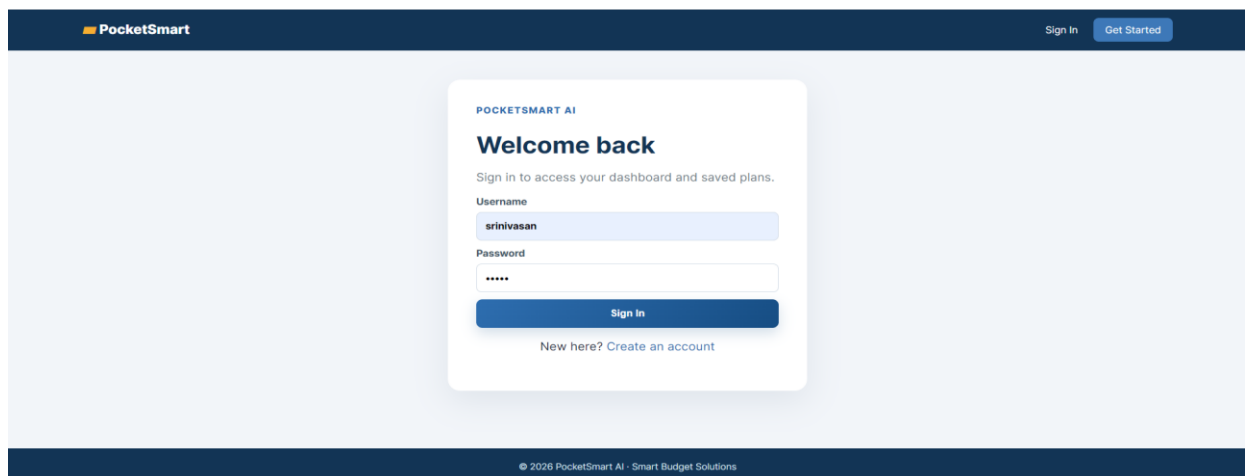
Description: The PocketSmart AI landing page introduces the application’s budget-aware planning concept and guides users toward the Home Interior, Party and Jewelry workflows. The responsive blue/white visual system is shared through the base template.



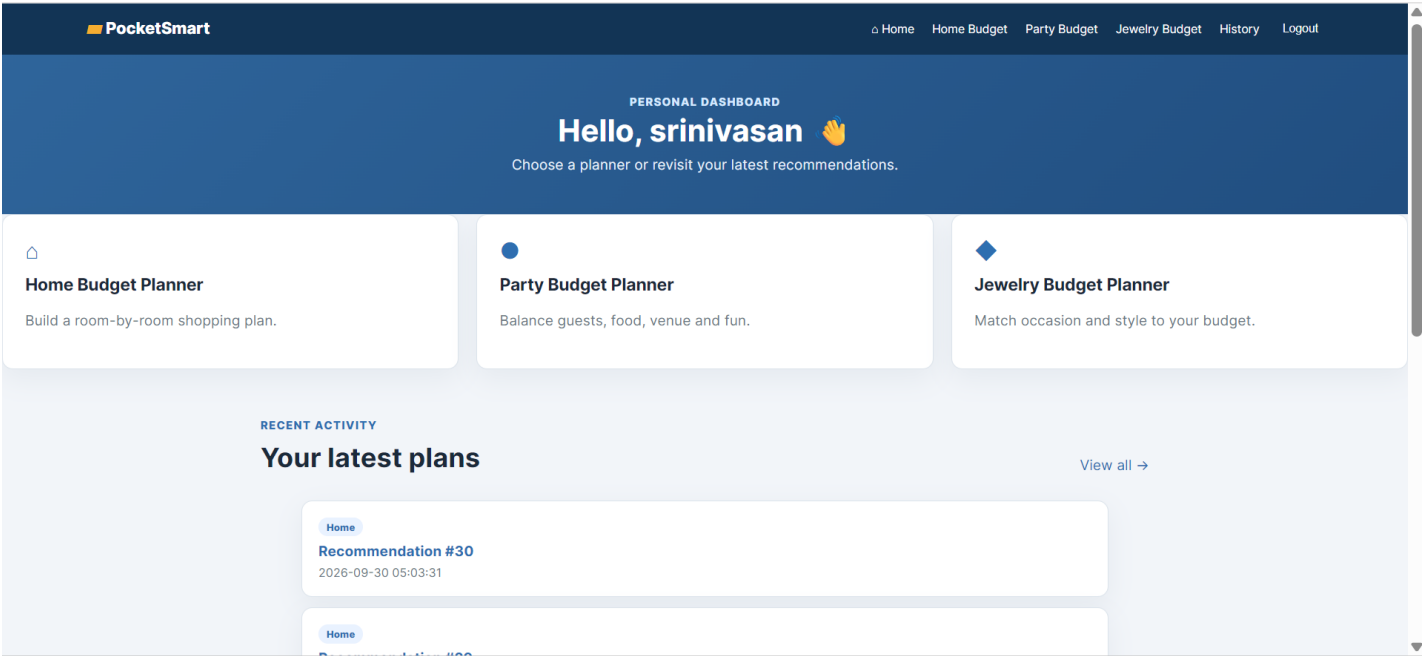
Login / Register Pages



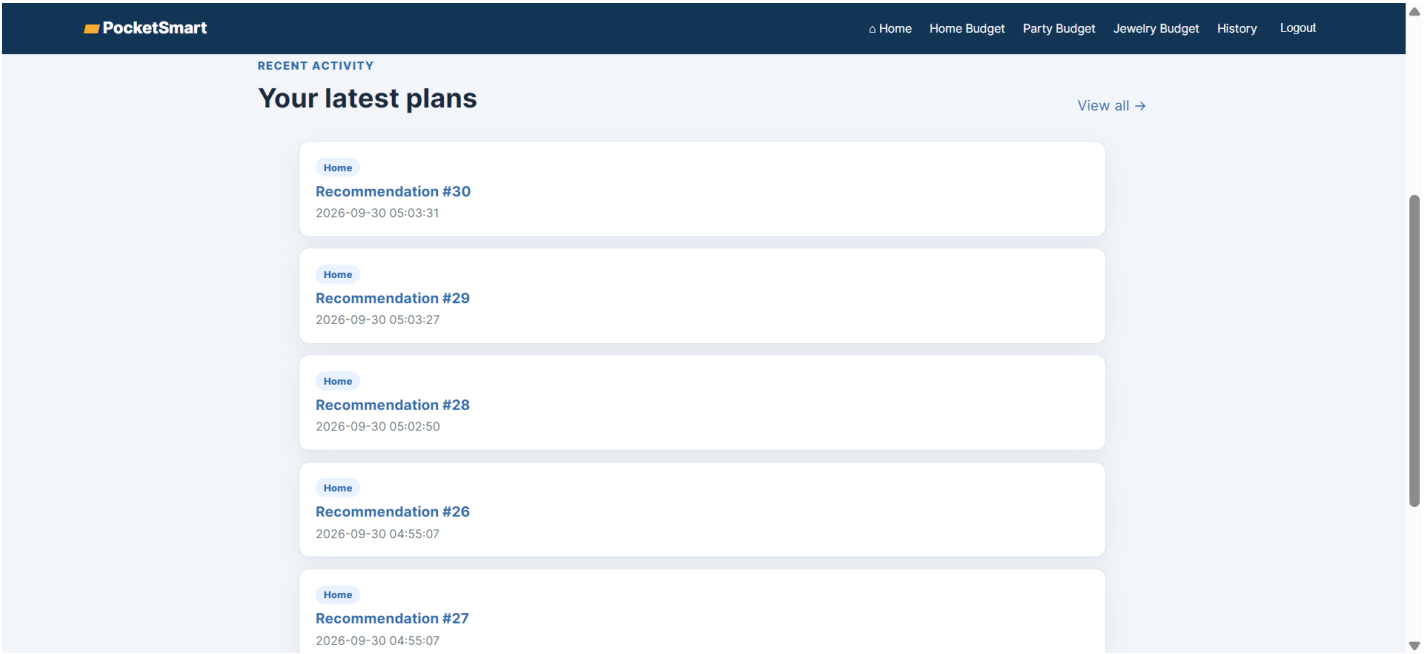
Description: Users create an account through Register and authenticate through Login. Passwords are stored using hashing rather than plain text, and authenticated sessions protect dashboard and history features.



Dashboard Page



Description: The dashboard acts as the main planning hub. It exposes the three planner modules and provides access to saved recommendation history for the authenticated user.



Navigation Flow

Home → Register/Login → Dashboard
Dashboard → Home Planner / Party Planner / Jewelry Planner
Planner → Generate Recommendation → Result
Result → Save → History → Recommendation Details

Exploring the Website’s Web Pages — Planner and Results

Home Planner Page

PocketSmart

[Home](#)[Home Budget](#)[Party Budget](#)[Jewelry Budget](#)[History](#)[Logout](#)

Home Interior Budget Planner

Plan your space with AI-powered budget recommendations.

🔥 Basic Information

Total Budget (₹)

Please fill out this field.

Room Type / Details

Living room, bedroom, kitchen...

💡 Room Requirements

Lighting quantity

1

Ceiling fan quantity

1

Furniture requirements

Sofa, chairs, table...

Table quantity

0

Style / Preferences

Description: The Home planner collects the total budget and home-specific requirements such as rooms, lighting, ceiling fans, furniture/table quantities and preferences. The result presents budget allocation, suggested items, indicative/demo prices, remaining budget and safe shopping/search links.

PocketSmart

[Home](#)[Home Budget](#)[Party Budget](#)[Jewelry Budget](#)[History](#)[Logout](#)

🔥 Room Requirements

Lighting quantity

1

Ceiling fan quantity

1

Furniture requirements

Sofa, chairs, table...

Table quantity

0

Style / Preferences

Modern, minimal, warm colors...

Additional requirements

Generate Recommendations

© 2026 PocketSmart AI · Smart Budget Solutions

After Generation:

1.

PocketSmart

Home

Home Budget

Party Budget

Jewelry Budget

History

Logout

HOME RECOMMENDATION

Your Personalized Budget Plan

Generated 2026-09-29 05:18:06

Total Budget

₹10000.00

Remaining Budget

₹3000.00

Lighting

Allocation: ₹500.00

Item	Description	Price	Qty	Shopping Links
LED Bulb (Warm White)	Energy-efficient lighting for general room use.	₹500.00	1	Amazon

Ceiling Fans

Allocation: ₹1800.00

Item	Description	Price	Qty	Shopping Links
Energy Efficient Ceiling Fan	Basic functional ceiling fan option.	₹1800.00	1	Flipkart

Furniture

2.

PocketSmart

Home

Home Budget

Party Budget

Jewelry Budget

History

Logout

Furniture

Allocation: ₹3500.00

Item	Description	Price	Qty	Shopping Links
Compact Table	Simple table suitable for dining or side use.	₹3500.00	1	IKEA

Decor

Allocation: ₹1200.00

Item	Description	Price	Qty	Shopping Links
Wall Art Set	Simple decorative accent for the selected room.	₹1200.00	1	Amazon

💡 Additional Suggestions

- Prioritize essential items first.
- Compare search results across platforms before buying.
- Keep a small buffer for delivery or installation costs.

Fallback demo recommendations are shown because a live Gemini response was unavailable.

© 2026 PocketSmart AI - Smart Budget Solutions

Party Planner Page

PocketSmart

HomeHome BudgetParty BudgetJewelry BudgetHistoryLogout

Party Budget Planner

Plan your event with AI-powered budget recommendations.

🔥 Basic Information

Total Budget (₹)

10000

Number of Guests

10

🎉 Event Details

Party Type

Birthday

Venue Type

Home

📋 Party Needs

☒ Catering

☒ Decoration

☒ Entertainment

Additional requirements

Themes, dietary restrictions, special requests...

Generate Budget Plan

Plan generated and saved.

Description: The Party planner collects budget, guest count, event type, venue, catering, decoration and entertainment preferences. The result organizes spending into event categories and keeps the generated total within the requested budget.

PocketSmart

HomeHome BudgetParty BudgetJewelry BudgetHistoryLogout

Total Budget

₹10000.00

Remaining Budget

₹0.00

Catering

Allocation: ₹5000.00

Item	Description	Price	Qty	Shopping Links
Catering package	Demo option for Birthday with 10 guests.	₹5000.00	1	Swiggy

Venue

Allocation: ₹2000.00

Item	Description	Price	Qty	Shopping Links
Budget-friendly venue/accommodation search	Demo option for Birthday with 10 guests.	₹2000.00	1	OYO

Decoration

Allocation: ₹1500.00

Item	Description	Price	Qty	Shopping Links
Party decoration set	Demo option for Birthday with 10 guests.	₹1500.00	1	Amazon

Entertainment

Allocation: ₹1500.00

Item	Description	Price	Qty	Shopping Links
Portable party speaker	Demo option for Birthday with 10 guests.	₹1500.00	1	Amazon

💡 Additional Suggestions

- Confirm venue capacity before booking.
- Compare per-guest catering rates.
- Keep a contingency buffer for last-minute needs.

Fallback demo recommendations are shown because a live Gemini response was unavailable.

Jewellery Planner Page

PocketSmart

HomeHome BudgetParty BudgetJewelry BudgetHistoryLogout

Jewelry Budget Planner

Get AI-powered jewelry recommendations within your budget.

Budget Details

Total Budget (₹)

Occasion & Preferences

Occasion

Birthday

Style Preferences

Gold, silver, minimal, traditional, statement...

Outfit / Style Description

Describe colors, material or style if you do not upload an image.

Upload Outfit Image

Choose FileNo file chosen

Optional - JPG/PNG/WEBP - max 5 MB

☒ Outfit preview

Get Recommendations

Description: The Jewelry planner collects occasion, style and outfit description, with an optional JPG/PNG/WEBP image upload. The result provides matching jewelry recommendations while applying the requested budget constraint.

PocketSmart

HomeHome BudgetParty BudgetJewelry BudgetHistoryLogout

JEWELRY RECOMMENDATION

Your Personalized Budget Plan

Generated 2026-09-29 05:20:19

Total Budget

₹100000.00

Remaining Budget

₹96700.00

Jewelry

Allocation: ₹2500.00

Item	Description	Price	Qty	Shopping Links
Classic Jewelry Set	Style-matched demo suggestion for Wedding; preference: gold.	₹2500.00	1	Amazon

Accessories

Allocation: ₹800.00

Item	Description	Price	Qty	Shopping Links
Matching Earrings	Coordinates with a broad range of outfit colors.	₹800.00	1	Flipkart

Additional Suggestions

- Match metal tone with the outfit and occasion.
- Use the image as a style/color reference, not as an identity source.
- Compare prices before purchase.

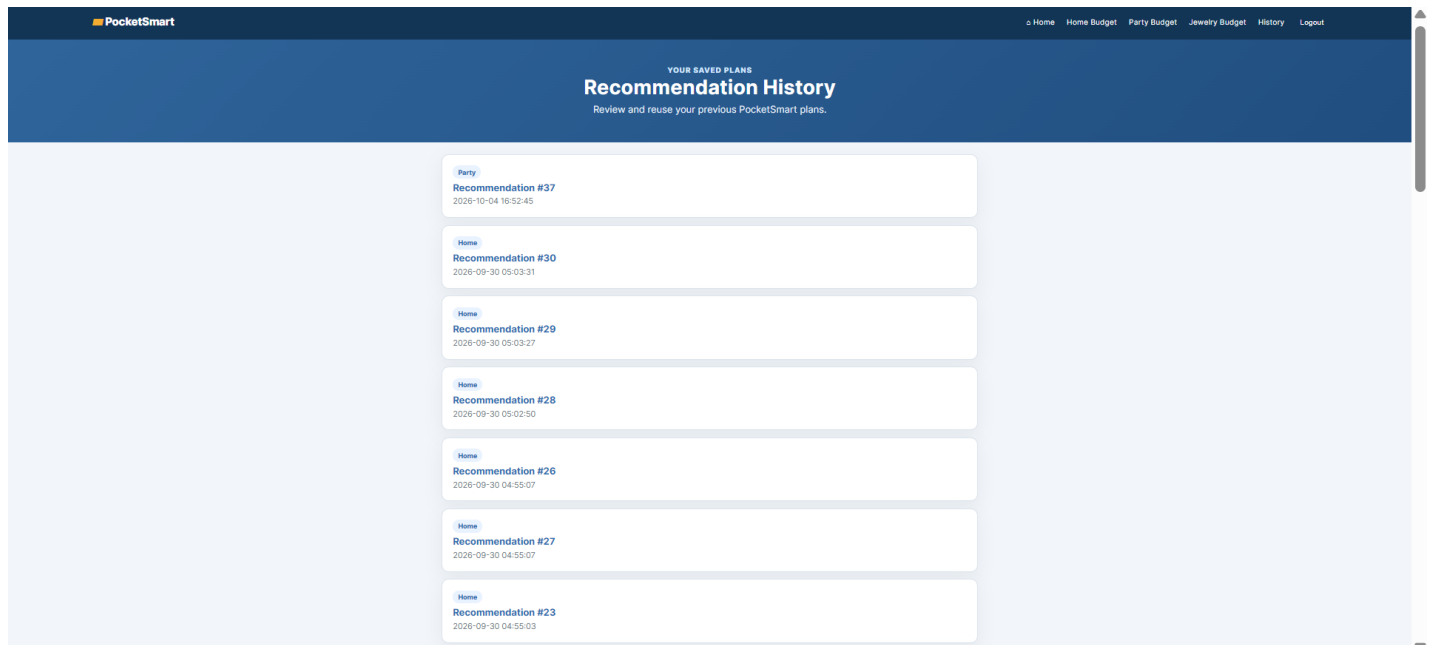
Fallback demo recommendations are shown because a live Gemini response was unavailable.

© 2026 PocketSmart AI - Smart Budget Solutions

Recommendation Result / Details

Description: The result view presents structured AI output in a readable format. Recommendation details can be revisited from History, allowing users to compare or reuse earlier planning sessions.

History Page



Description: The History page lists recommendations associated with the authenticated user. Selecting an entry opens the detailed recommendation view.

Conclusion:

PocketSmart AI is a Generative AI budget-planning platform built with FastAPI and Jinja2 that helps users organize Home Interior, Party and Jewelry requirements around a defined budget. It combines authentication, structured validation, SQLite persistence, Gemini-based recommendation generation, optional outfit-image analysis and safe external search links into a single student-friendly web application.

The project workflow covers model configuration, architecture design, core planner development, backend validation, responsive frontend development, automated testing and public deployment. A key implementation principle is that generated recommendations are treated as untrusted model output: Pydantic validation and backend budget normalization are applied before results are displayed.

The application is designed to remain usable when Gemini is unavailable by providing clearly labelled demo fallback recommendations. It also avoids claiming real-time inventory from external platforms; Amazon, Flipkart, IKEA, Swiggy, Zomato and OYO are represented through safe search/demo links.

Looking ahead, the project can evolve through verified real-time product/service integrations, richer multimodal analysis, stronger recommendation evaluation, managed database infrastructure, improved personalization and broader deployment scalability while retaining the same budget-aware planning workflow.